actual states.

If you consider the four principles, you'll notice that we've applied the first two already. We expressed the desired state for our applications declaratively using Kubernetes manifests and Kustomize. And we stored the desired state in a Git repository on GitHub (`polar-deployment`), making it versioned and immutable. We are still missing a software agent that automatically pulls the desired state declarations from the Git source and continuously reconciles them inside the Kubernetes cluster, therefore achieving continuous deployment.

We'll start by installing Argo CD (https://argo-cd.readthedocs.io), a GitOps software agent. Then we'll configure it to complete the final step of the deployment pipeline and let it monitor our `polar-deployment` repository. Whenever there's a change in the application manifests, Argo CD will apply the changes to our production Kubernetes cluster.

15.4.1 Implementing GitOps with Argo CD

Let's start by installing the Argo CD CLI. Refer to the project website for installation instructions (https://argo-cd.readthedocs.io). If you are on macOS or Linux, you can use Homebrew as follows:

```
$ brew install argocd
```

We'll use the CLI to instruct Argo CD about which Git repository to monitor, and we'll configure it to apply changes to the cluster to achieve continuous deployment automatically. But first we need to deploy Argo CD to the production Kubernetes cluster.

NOTE I'll assume your Kubernetes CLI is still configured to access the production cluster on DigitalOcean. You can check that with `kubectl config current-context`. If you need to